

โครงงาน

Fishing Game

จัดทำโดย

6804062610207 นางสาวฐิติชญา ปานงาม

เสนอ

ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์

รายวิชา Object-Oriented Programming

ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

สารบัญ

บทที่ 1 บทนำ 1

ที่มาและความสำคัญของโครงการ 1

ประเภทของโครงการ 1

ประโยชน์ที่ได้รับจากโครงการ 1

ขอบเขตของโครงการ 2

บทที่ 2 ส่วนการพัฒนา 3

2.1 เนื้อเรื่องย่อและวิธีการเล่น 3

2.2 คลาสไดอะแกรมและคำอธิบาย 4

2.3 รูปแบบการพัฒนา 5

2.4 การประยุกต์ใช้หลักการ OOP ในโปรแกรม 6

2.5 ส่วนของหน้าจอ GUI 9

2.6 อัลกอริทึมที่สำคัญในโปรแกรม 13

บทที่ 3 สรุป 16

3.1 ปัญหาที่พบระหว่างการพัฒนา 16

3.2 จุดเด่นของโปรแกรม 16

3.3 คำแนะนำ / ข้อเสนอแนะ 17

ชื่อโครงงาน: Fishing Game

จัดทำโดย: นางสาวฐิติชญา ปานงาม

อาจารย์ผู้สอน: ผู้ช่วยศาสตราจารย์สถิต ประสมพันธ์

Source Code: https://github.com/nicethiti/FishingGame.git

บทที่ 1 บทนำ

ที่มาและความสำคัญของโครงการ

โครงงานนี้จัดทำขึ้นเพื่อประเมินความรู้ความเข้าใจในรายวิชา Object-Oriented Programming โดยนำหลักการเขียนโปรแกรมเชิงวัตถุมาประยุกต์สร้างเป็นเกมตกปลา (Fishing Game) ซึ่งเป็นเกมที่มีวัตถุหลายประเภททำงานร่วมกัน ได้แก่ ผู้เล่น ปลา และสิ่งกีดขวาง เหมาะแก่การแสดงแนวคิดเรื่องคลาส การสืบทอด และความสัมพันธ์ระหว่างอ็อบเจกต์ นอกจากนี้ ผู้จัดทำยังได้ศึกษา เครื่องมือ สำหรับ พัฒนาเกม บนภาษา Java ได้แก่ Java Swing, AWT และ Graphics2D รวมถึงแนวคิดลูปของเกม (Game Loop) และการจัดการเหตุการณ์จากแป้นพิมพ์และเมาส์

ประเภทของโครงการ

โครงงานพัฒนาเกม (Game Development Project)

ประโยชน์ที่ได้รับจากโครงการ

- ฝึกการวิเคราะห์และออกแบบโปรแกรมด้วยแนวคิดเชิงวัตถุ
- ฝึกการใช้ Java Swing และ Graphics2D ในการสร้างส่วนติดต่อผู้ใช้และกราฟิก

- ฝึกการเขียนโปรแกรมแบบ Event-driven และการควบคุมเวลาด้วย Timer
- ผู้เล่นได้ฝึกสมาธิ การตัดสินใจ และการประสานสายตากับมือ ควบคู่กับความบันเทิง

ขอบเขตของโครงการ

ความต้องการของระบบ (Functional Requirements)

- มีหน้าจอเริ่มต้น (Home) แสดงวิธีเล่นและปุ่ม Start
- ผู้เล่นเคลื่อนที่ซ้าย–ขวาและปล่อยเบ็ดลงน้ำเพื่อจับปลาได้
- ปลามีหลายชนิด (Small Fish, Big Fish) เคลื่อนที่อัตโนมัติและให้คะแนนต่างกัน
- มีสิ่งกีดขวาง 3 ชนิด (Rock, Trash, Mine) ซึ่งหักคะแนนเมื่อเบ็ดชน
- มี 3 ด่าน (Shallow Pond, River, Deep Sea) ความยากเพิ่มขึ้นตามด่าน
- แสดงด่าน จำนวนปลาที่จับได้ คะแนน และเวลาที่เหลือ
- ตัดสินแพ้เมื่อหมดเวลา ชนะเมื่อผ่านครบทุกด่าน และกลับหน้า Home ได้ด้วยปุ่ม R

ตารางเวลาแผนการดำเนินโครงการ (Project Schedule)

ลำดับ	รายการ	14 - 17	18 - 25	26 - 31
1	ศึกษาเอกสารและข้อมูลที่เกี่ยวข้อง			
2	ออกแบบคลาสและกราฟิก			
3	ลงมือเขียนโปรแกรม			
4	จัดทำเอกสาร			
5	ตรวจสอบและแก้ไขข้อผิดพลาด			

บทที่ 2 ส่วนการพัฒนา

2.1 เนื้อเรื่องย่อและวิธีการเล่น

เนื้อเรื่องย่อ

ผู้เล่นรับบทเป็นนักตกปลาที่ยืนอยู่บนฝั่งต้องปล่อยเบ็ดลงน้ำเพื่อจับปลาให้ครบตามเป้าหมายของแต่ละด่านก่อนเวลาจะหมด โดยต้องระวังไม่ให้เบ็ดไปโดนหินใต้น้ำ ขยะ และทุ่นระเบิด ซึ่งจะทำให้เสียคะแนน เกมมี 3 ด่าน คือ Shallow Pond, River และ Deep Sea

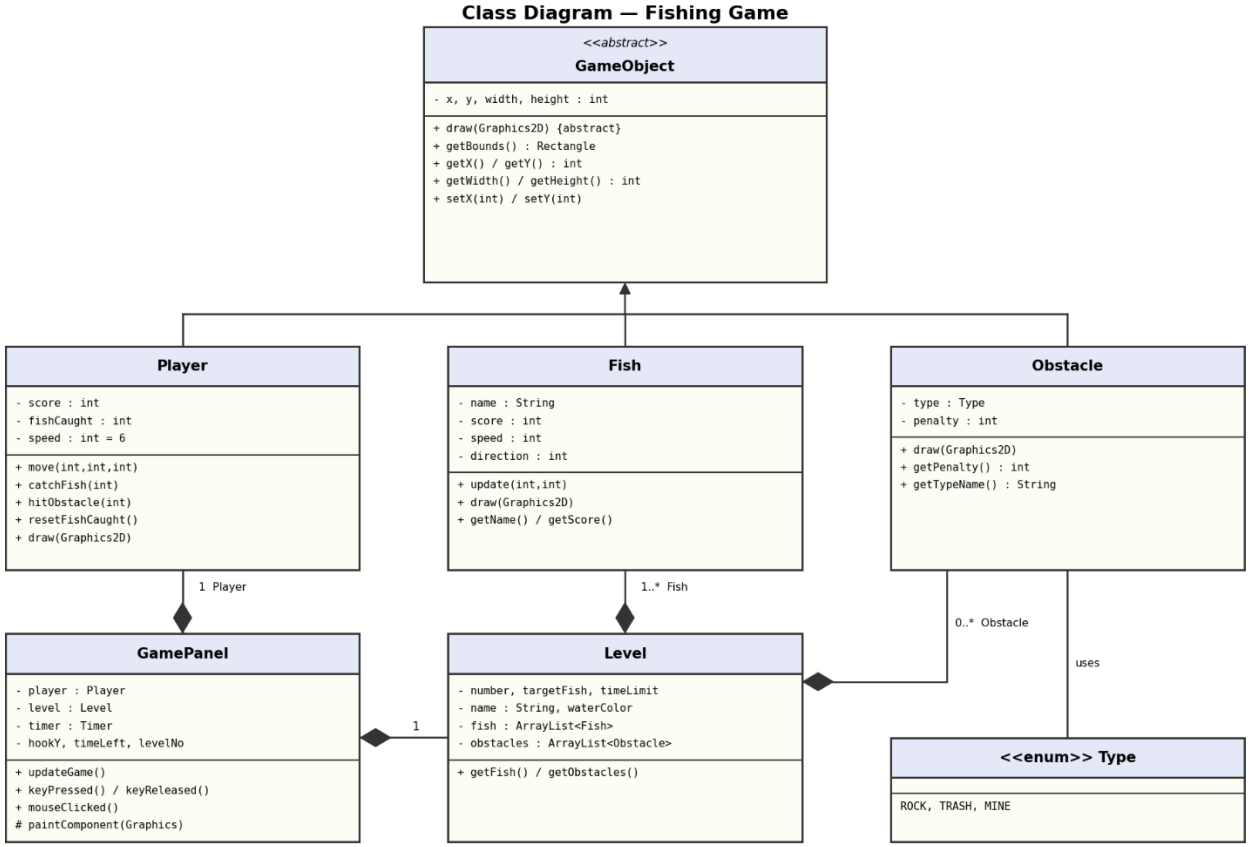
วิธีการเล่น

1. หน้า Home คลิกปุ่ม Start หรือกด Enter / Space เพื่อเริ่มเกมที่ด่าน 1
2. กดปุ่มลูกศรซ้าย–ขวา (⟵ ⟶) เพื่อเคลื่อนที่ตัวละครไปตามแนวนอน (กดค้างเพื่อเคลื่อนที่ต่อเนื่อง)
3. กด Space เพื่อปล่อยเบ็ดลงน้ำ เบ็ดจะจมลงในแนวดิ่งจากตำแหน่งของผู้เล่น
4. หากเบ็ดชนปลา จะได้คะแนน (Small Fish 10 คะแนน, Big Fish 20 คะแนน) และเบ็ดถูกดึงกลับ
5. หากเบ็ดชนสิ่งกีดขวาง จะถูกหักคะแนน (Rock 10, Trash 15, Mine 30) โดยคะแนนไม่ติดลบ
6. จับปลาให้ครบเป้าหมายก่อนเวลาหมดเพื่อไปด่านถัดไป หากผ่านด่าน 3 จะชนะเกม
7. หากเวลาหมดก่อนจับปลาครบ จะแพ้ (Time out!) กด R เพื่อกลับหน้า Home

ความแตกต่างของแต่ละด่าน

รายการ	ด่าน 1: Shallow Pond	ด่าน 2: River	ด่าน 3: Deep Sea
เป้าหมายปลา	5 ตัว	7 ตัว	9 ตัว
เวลา	45 วินาที	50 วินาที	55 วินาที
Small Fish / Big Fish	4 / 2	5 / 3	6 / 4
ความเร็ว Small / Big	3 / 2	4 / 3	5 / 4
สิ่งกีดขวาง	Rock 2	Rock 2, Trash 1	Rock 1, Trash 1, Mine 2

2.2 คลาสไดอะแกรมและคำอธิบาย



รูปที่ 1 Class Diagram ของโปรแกรม

โปรแกรมประกอบด้วยคลาสที่เขียนขึ้นเอง 7 คลาส (รวม GamePanel ที่เป็น static nested class ภายใน FishingGame) และ enum 1 ตัว ดังนี้

คลาส	ประเภท	หน้าที่
GameObject	abstract class	คลาสแม่ของวัตถุในเกม เก็บตำแหน่งและขนาด มีเมธอด getBounds() สำหรับตรวจการชน และประกาศ draw() เป็น abstract
Player	extends GameObject	ผู้เล่น เก็บคะแนนและจำนวนปลาที่จับได้ มีเมธอดเคลื่อนที่ จับปลา และถูกหักคะแนน

คลาส	ประเภท	หน้าที่
Fish	extends GameObject	ปลา เคลื่อนที่ไป–มาอัตโนมัติและกลับทิศเมื่อถึงขอบ
Obstacle	extends GameObject	สิ่งกีดขวาง มี enum Type (ROCK, TRASH, MINE) กำหนดค่าปรับและรูปร่างที่วาด
Level	คลาสทั่วไป	กำหนดค่าของแต่ละด่าน (ชื่อ สีน้ำ เป้าหมาย เวลา) และสร้างปลากับสิ่งกีดขวางประจำด่าน
FishingGame	extends JFrame	หน้าต่างหลักและจุดเริ่มโปรแกรม (main)
GamePanel	static nested, extends JPanel	คลาสหลักของเกม ควบคุม Game Loop ตรรกะเบ็ด การตรวจชน การวาดหน้าจอ และรับอินพุตทั้งหมด

2.3 รูปแบบการพัฒนา

- **ภาษา:** Java (แพ็กเกจ fishinggame)
- **รูปแบบโปรแกรม:** Application แบบ Desktop Standalone (ไม่ใช่ Applet) เริ่มทำงานที่ main() ของ FishingGame ซึ่งสร้างหน้าต่าง JFrame ผ่าน SwingUtilities.invokeLater() และส่งออกเป็นไฟล์ .jar
- **เครื่องมือ/ไลบรารี:** Visual Studio Code, Java Swing (javax.swing), AWT (java.awt, java.awt.event, java.awt.geom) และ Graphics2D สำหรับวาดภาพ
- **สถาปัตยกรรม:** แยกข้อมูลและพฤติกรรมของวัตถุในเกม (Player, Fish, Obstacle, Level) ออกจากส่วนควบคุมและแสดงผล (GamePanel) โดยสืบทอดจาก GameObject เพื่อให้ทุกวัตถุมีโครงสร้างร่วมกัน
- **Game Loop:** ใช้ javax.swing.Timer ทำงานทุก 40 มิลลิวินาที (ประมาณ 25 เฟรมต่อวินาที)
- **Assets:** ไม่ใช้ไฟล์ภาพภายนอก ภาพทั้งหมดวาดด้วยโค้ด Graphics2D

2.4 การประยุกต์ใช้หลักการ OOP ในโปรแกรม

Constructor

Constructor ใช้กำหนดค่าเริ่มต้นให้อ็อบเจกต์ทันทีที่สร้าง โปรแกรมมี constructor ดังนี้

- GameObject(x, y, width, height) กำหนดตำแหน่งและขนาด
- Player(x, y) เรียก super() ส่งขนาดคงที่ 42×48
- Fish(name, x, y, score, speed) เรียก super() โดยเลือกขนาดตามชนิดปลา (Big Fish ใหญ่กว่า)
- Obstacle(type, x, y) กำหนดค่าปรับตามชนิดสิ่งกีดขวาง
- Level(number) เลือกค่าด่านและสร้างปลา/สิ่งกีดขวางทั้งหมดของด่านนั้น
- GamePanel() ตั้งขนาดหน้าจอ ผูก Listener เริ่มด่านแรก และสร้าง Timer ของเกม
- FishingGame() ตั้งชื่อ ขนาด ตำแหน่ง และเพิ่ม GamePanel ลงในหน้าต่าง

```
public Fish(String name, int x, int y, int score, int speed) {
    super(x, y, name.equals("Big Fish") ? 42 : 30, name.equals("Big Fish") ? 22 : 16);
    this.name = name;
    this.score = score;
    this.speed = speed;
}

public Obstacle(Type type, int x, int y) {
    super(x, y, 38, 30);
    this.type = type;
    this.penalty = type == Type.MINE ? 30 : (type == Type.TRASH ? 15 : 10);
}
```

Encapsulation

ฟิลด์ของคลาสเกือบทั้งหมดประกาศเป็น private เช่น x, y, width, height ใน GameObject, score, fishCaught ใน Player และ name, score, speed ใน Fish ภายนอกต้องเข้าถึงผ่านเมธอด public เช่น getScore(), getX(), setX() ฟิลด์ที่ไม่ควรเปลี่ยนค่าหลังสร้างประกาศเป็น final เช่น ใน Level (number, name, targetFish, timeLimit) และใน Obstacle (type, penalty) นอกจากนี้ getBounds() สร้างอ็อบเจกต์ Rectangle ใหม่ทุกครั้ง จึงไม่เปิดเผยตัวแปรภายในโดยตรง

```java
public class Player extends GameObject {
    private int score;
    private int fishCaught;
    private final int speed = 6;

    public void catchFish(int points) { fishCaught++; score += points; }
    public void hitObstacle(int penalty) { score = Math.max(0, score - penalty); }
    public int getScore() { return score; }
}
```

เมธอด hitObstacle() เป็นตัวอย่างที่ดี เพราะกฎ "คะแนนต้องไม่ติดลบ" ถูกบังคับอยู่ภายในคลาส Player ผู้เรียกใช้ไม่สามารถทำให้คะแนนผิดกติกาได้

Composition

ความสัมพันธ์แบบ "has-a" ที่พบในโปรแกรม ได้แก่ FishingGame มี GamePanel, GamePanel มี Player และ Level, และ Level มีรายการ Fish และ Obstacle เป็นส่วนประกอบ ซึ่งถูกสร้างภายใน constructor ของ Level เอง เมื่อเปลี่ยนด่าน GamePanel จะสร้าง Level ใหม่ ทำให้ปลาและสิ่งกีดขวางของด่านเก่า หมดอายุไปพร้อมกัน

```java
public class Level {
    private final ArrayList<Fish> fish = new ArrayList<>();
    private final ArrayList<Obstacle> obstacles = new ArrayList<>();
    ...
}

// in GamePanel
private Player player = new Player(430, 60);
private Level level = new Level(1);
```

Polymorphism

คลาสลูกทั้งสามชนิด (Player, Fish, Obstacle) override เมธอด draw() ของ GameObject ให้วาดภาพต่างกัน เมื่อ GamePanel เรียก o.draw(g), f.draw(g) หรือ player.draw(g) ระบบจะเลือกเมธอดตามชนิดอ็อบเจกต์จริงขณะรันไทม์ (dynamic binding) นอกจากนี้ Obstacle.draw() ยังแตกพฤติกรรมตามค่า Type และ GamePanel เขียนทับเมธอด paintComponent(), keyPressed(), keyReleased() และ mouseClicked() ตามสัญญาของ JPanel, KeyListener และ MouseListener

```
// GameObject: declaration
public abstract void draw(Graphics2D g);

// Fish: override
@Override
public void draw(Graphics2D g) { g.fillOval(getX(), getY(), getWidth(), getHeight()); ... }

// GamePanel: usage
for (Obstacle o : level.getObstacles()) o.draw(g);
for (Fish f : level.getFish()) f.draw(g);
player.draw(g);
```

Abstract

GameObject เป็น abstract class ที่ผู้จัดทำออกแบบเอง ไม่สามารถสร้างอ็อบเจกต์จากมันโดยตรงได้ (ไม่มีความหมายว่าเป็นวัตถุที่วาดไม่ได้) โดยเก็บข้อมูลร่วม (ตำแหน่ง ขนาด) และเมธอดร่วม (getBounds()) พร้อมประกาศ draw() เป็น abstract method บังคับให้คลาสลูกทุกตัวต้องนิยามวิธีวาดของตนเอง นอกจากนี้ยังใช้ interface ของ Java ได้แก่ KeyListener และ MouseListener ใน GamePanel

```
public abstract class GameObject {
    private int x, y, width, height;
    public abstract void draw(Graphics2D g);
    public Rectangle getBounds() { return new Rectangle(x, y, width, height); }
}
```

Inheritance

มีการสืบทอด 2 ระดับ คือ (1) คลาสที่ผู้จัดทำเขียนเอง Player, Fish และ Obstacle สืบทอดจาก GameObject เพื่อใช้ตำแหน่ง ขนาด และ getBounds() ร่วมกัน ลดโค้ดซ้ำซ้อน และ (2) สืบทอดจากคลาสของ Swing ได้แก่ FishingGame extends JFrame และ GamePanel extends JPanel

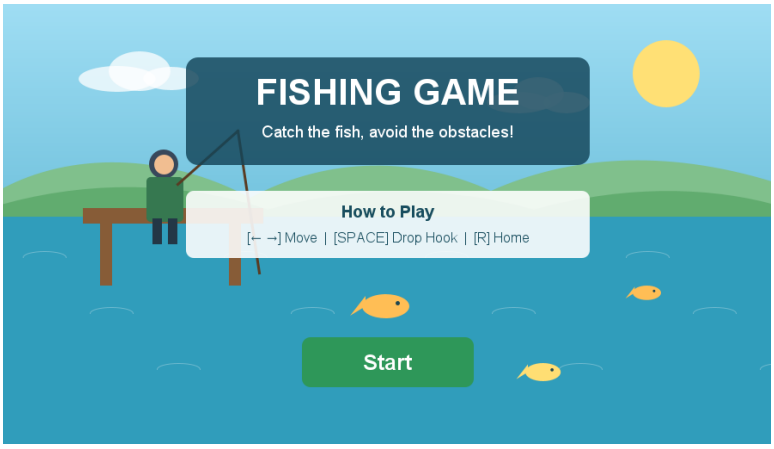
```
public class Player extends GameObject { ... }
public class Fish extends GameObject { ... }
public class Obstacle extends GameObject { ... }
public class FishingGame extends JFrame { ... }
static class GamePanel extends JPanel implements KeyListener, MouseListener { ... }
```

2.5 ส่วนของหน้าจอ GUI

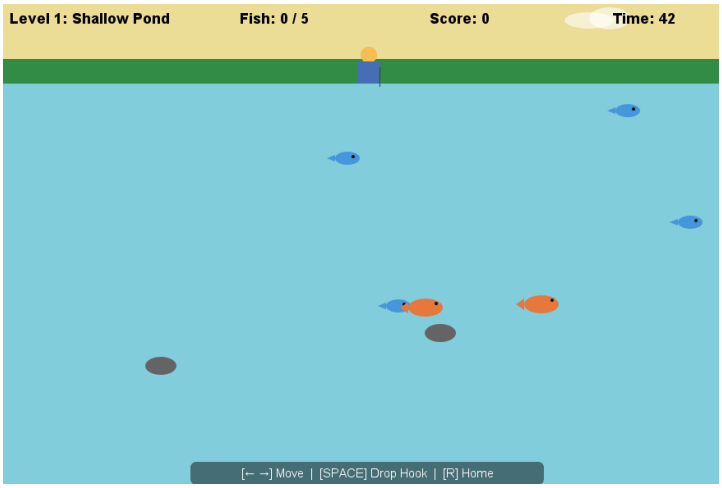
โครงสร้างและ Component

หน้าต่างหลักคือ JFrame (FishingGame) ขนาดคงที่ ปรับขนาดไม่ได้ (setResizable(false)) แสดงกลางจอ (setLocationRelativeTo(null)) ภายในมี GamePanel เพียงตัวเดียวขนาด 900×600 พิกเซล โปรแกรมไม่ใช้ Swing component มาตรฐานอย่าง JButton หรือ JLabel แต่วาดทุกส่วนด้วย Graphics2D ภายใน paintComponent() (Custom Painted GUI) และตรวจจับการคลิกด้วยการเทียบพิกัดกับ Rectangle ชื่อ startButton หน้าจอที่แสดงขึ้นอยู่กับตัวแปรสถานะของ GamePanel ดังนี้

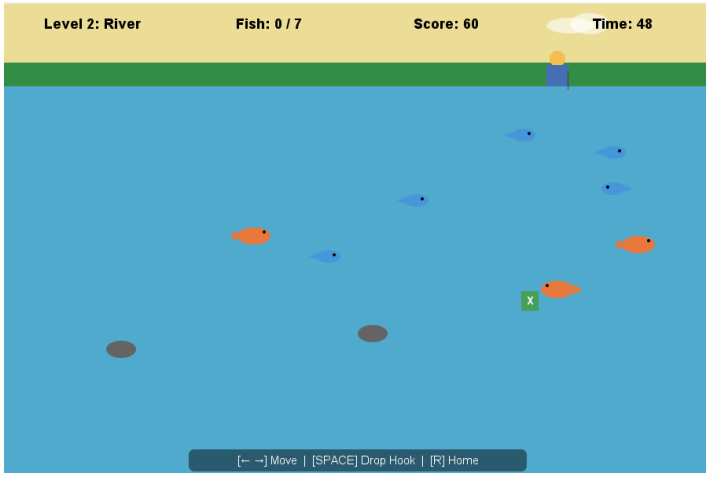
สถานะ	สิ่งที่วาดบนหน้าจอ
หน้า Home (started = false)	พื้นหลังไล่สี ท้องฟ้า เนินเขา ทะเล ตัวละครนั่งตกปลา ปลาประดับ แผ่นป้ายชื่อเกม FISHING GAME กล่องวิธีเล่น (How to Play) และปุ่ม Start ที่เปลี่ยนสีเมื่อเมาส์วางทับ (hover)
หน้าเล่นเกม (started = true)	พื้นหลังสีน้ำตามด่าน ชายฝั่งและหญ้า สิ่งกีดขวาง ปลา ตัวผู้เล่น เส้นเบ็ดและเบ็ด แถบ HUD ด้านบน (Level, Fish, Score, Time) ข้อความแจ้งเตือนชั่วคราว และแถบคำแนะนำปุ่มด้านล่าง
หน้าจบเกม (gameOver หรือ win)	กล่องโปร่งแสงวางทับ แสดงข้อความ "You win!" หรือ "Time out!" คะแนนรวม และข้อความ "Press R for Home"



รูปที่ 2 หน้าจอเริ่มต้น



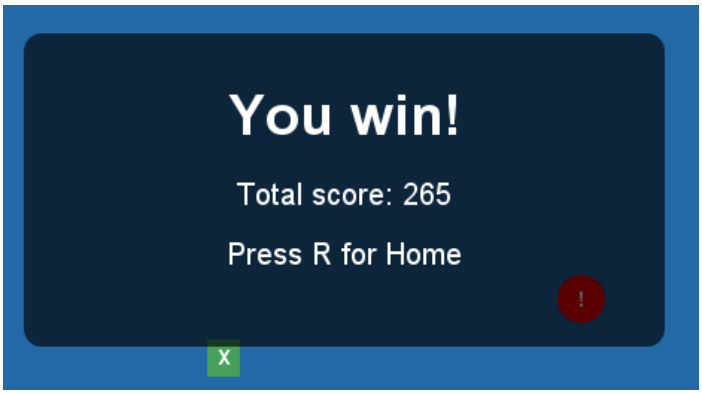
รูปที่ 3 หน้าเล่นเกม ด่านที่ 1



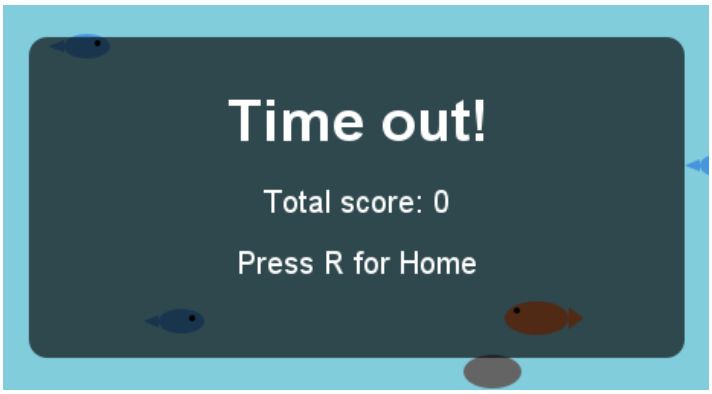
รูปที่ 4 หน้าเล่นเกม ด่านที่ 2



รูปที่ 5 หน้าเล่นเกม ด่านที่ 3



รูปที่ 6 หน้าจบเกมแบบที่ 1



รูปที่ 7 หน้าจบเกมแบบที่ 2

Event Handling

Event	การทำงาน
keyPressed() (KeyListener)	หน้า Home: Enter/Space เริ่มเกม; ขณะเล่น: ← → ตั้งค่า leftPressed/rightPressed, Space ปล่อยเบ็ด (ถ้าเบ็ดยังไม่ถูกปล่อย), R กลับหน้า Home
keyReleased() (KeyListener)	เคลียร์ค่า leftPressed/rightPressed เมื่อปล่อยปุ่ม ทำให้ตัวละครเคลื่อนที่ต่อเนื่องขณะกดค้าง
mouseClicked() (MouseListener)	หน้า Home: ตรวจว่าพิกัดคลิกอยู่ใน startButton หรือไม่ ถ้าใช่เรียก startGame()
Timer ทุก 40 ms (ActionListener แบบ lambda)	Game Loop หลัก: เรียก updateGame() เพื่อเคลื่อนที่ผู้เล่นและปลา อัปเดตเบ็ด ตรวจการชน ตรวจความคืบหน้าด่าน นับเวลา แล้วสั่ง repaint()
การตรวจจับการชน (Collision)	updateHook() ใช้ Rectangle.intersects() ระหว่างกรอบของเบ็ดกับ getBounds() ของปลาและสิ่งกีดขวาง

```
timer = new Timer(40, e -> updateGame());
timer.start();

@Override
public void mouseClicked(MouseEvent e) {
    if (!started && startButton.contains(e.getPoint())) startGame();
}
```

2.6 อัลกอริทึมที่สำคัญในโปรแกรม

1) Game Loop

Timer ทำงานทุก 40 มิลลิวินาที เรียก updateGame() ซึ่งทำงานตามลำดับ คือ เคลื่อนที่ผู้เล่น ⟶ อัปเดตตำแหน่งปลาทุกตัว ⟶ อัปเดตเบ็ดและตรวจชน (ถ้าปล่อยเบ็ด) ⟶ ตรวจความคืบหน้าด่าน ⟶ นับเวลาถอยหลัง ⟶ ลดตัวนับข้อความ ⟶ repaint() การอัปเดตจะหยุดเมื่อเกมยังไม่เริ่มหรือจบแล้ว แต่ยัง repaint() ต่อเพื่อให้หน้าจอ (เช่น hover ของปุ่ม Start) ทำงานได้

```
void updateGame() {
    if (started && !gameOver && !win) {
        movePlayer();
        for (Fish f : level.getFish()) f.update(40, W - 40);
        if (hook) updateHook();
        checkLevelProgress();
        updateTimer();
        if (messageTicks > 0) messageTicks--;
    }
    repaint();
}
```

2) การเคลื่อนที่ของปลา (Bouncing Movement)

ปลาแต่ละตัวมีความเร็ว (speed) และทิศทาง (direction = +1 หรือ −1) ทุกเฟรมจะเลื่อนตำแหน่ง x ไปเท่ากับ speed × direction และเมื่อชนขอบซ้ายหรือขวาของช่วงที่กำหนด (40 ถึง 860) จะกลับทิศทางด้วยการคูณ −1 ความเร็วของปลาเพิ่มตามหมายเลขด่าน (Small Fish = 2 + ด่าน, Big Fish = 1 + ด่าน) จึงเป็นกลไกเพิ่มความยาก ตำแหน่งเริ่มต้นสุ่มด้วย Random ที่กำหนด seed จากหมายเลขด่าน ทำให้ด่านเดียวกันมีการวางปลาเหมือนเดิมทุกครั้ง

```
public void update(int minX, int maxX) {
    setX(getX() + speed * direction);
    if (getX() <= minX || getX() + getWidth() >= maxX) direction *= -1;
}
```

3) การเคลื่อนที่ของผู้เล่น

คำนวณทิศทางจากสถานะปุ่ม dir = (rightPressed ? 1 : 0) − (leftPressed ? 1 : 0) แล้วเรียก player.move() ซึ่งเลื่อนตำแหน่งครั้งละ speed (6 พิกเซล) และจำกัดไม่ให้ออกนอกจอด้วย Math.max และ Math.min

```
public void move(int dx, int minX, int maxX) {
    setX(Math.max(minX, Math.min(maxX - getWidth(), getX() + dx * speed)));
}
```

4) ระบบเบ็ดและการตรวจจับการชน (Collision Detection)

เมื่อกด Space เบ็ดจะจมลงด้วยความเร็ว 5 พิกเซลต่อเฟรม จาก WATER_TOP จนถึง HOOK_MAX_Y (560) โปรแกรมสร้างสี่เหลี่ยม (hit box) กว้าง 14 พิกเซล ครอบคลุมตั้งแต่ผิวน้ำถึงปลายเบ็ด แล้วใช้ Rectangle.intersects() ตรวจกับปลาทุกตัวก่อน หากชนจะเรียก catchFish() ลบปลาออกจากลิสต์ด้วย Iterator.remove() รีเซ็ตเบ็ด และแสดงข้อความ หากไม่ชนปลาจึงตรวจกับสิ่งกีดขวาง และเรียก hitObstacle() หากเบ็ดจมถึงก้นน้ำโดยไม่ชนสิ่งใดจะรีเซ็ตเบ็ดกลับ การใช้ Iterator.remove() ช่วยหลีกเลี่ยงข้อผิดพลาด ConcurrentModificationException จากการลบสมาชิกระหว่างวนลูป

```
Rectangle hb = new Rectangle(player.getX() + player.getWidth() / 2 - HOOK_WIDTH / 2,
        WATER_TOP, HOOK_WIDTH, Math.max(1, hookY - WATER_TOP));
for (Iterator<Fish> it = level.getFish().iterator(); it.hasNext(); ) {
    Fish f = it.next();
    if (hb.intersects(f.getBounds())) {
        player.catchFish(f.getScore());
        it.remove();
        resetHook();
        return;
    }
}
```

5) ระบบคะแนน

ปลาเล็กได้ 10 คะแนน ปลาใหญ่ได้ 20 คะแนน (กำหนดตอนสร้าง Fish ใน Level.addFish()) ส่วนสิ่งกีดขวางหักคะแนนตามชนิด คือ Rock 10, Trash 15, Mine 30 โดยใช้ Math.max(0, score − penalty)

เพื่อไม่ให้คะแนนติดลบ จำนวนปลาที่จับได้ถูกนับแยกต่อด่าน (resetFishCaught() เมื่อขึ้นด่านใหม่) แต่คะแนนสะสมต่อเนื่องตลอดเกม

6) การนับเวลาและเงื่อนไขชนะ/แพ้

เวลาถอยหลังนับด้วย System.currentTimeMillis() เมื่อผ่านไปครบ 1,000 ms จะลด timeLeft ลง 1 หากถึง 0 เกมจบ (Time out!) ส่วน checkLevelProgress() เปรียบเทียบจำนวนปลาที่จับได้กับ level.getTargetFish() เมื่อครบเป้าหมายและยังไม่ใช่ด่านสุดท้าย จะเพิ่มหมายเลขด่านและสร้าง Level ใหม่ แต่ถ้าเป็นด่านที่ 3 จะตั้งค่า

```
private void updateTimer() {
    long now = System.currentTimeMillis();
    if (now - lastSecond >= 1000) {
        lastSecond = now;
        timeLeft--;
        if (timeLeft <= 0) gameOver = true;
    }
}
```

บทที่ 3 สรุป

3.1 ปัญหาที่พบระหว่างการพัฒนา

- **การตรวจจับการชนของเบ็ด:** เบ็ดเป็นเส้นยาวจากผิวน้ำ จึงต้องใช้สี่เหลี่ยมที่ขยายตามความลึก แทนจุดเดียว และต้องกำหนดลำดับการตรวจให้ชัดเจน (ปลาก่อนสิ่งกีดขวาง) มิฉะนั้นอาจเกิดผลที่ไม่คาดคิด
- **การลบปลาออกจากลิสต์ระหว่างวนลูป:** ต้องใช้ Iterator.remove() เพื่อหลีกเลี่ยง ConcurrentModificationException
- **การรับอินพุตแบบต่อเนื่อง:** การเลื่อนตัวละครโดยตรงใน keyPressed() ให้การเคลื่อนที่กระตุก จึงแก้โดยเก็บสถานะปุ่มเป็น leftPressed/rightPressed แล้วประมวลผลในลูปเกม
- **การโฟกัสแป้นพิมพ์:** ต้องเรียก setFocusable(true) และ requestFocusInWindow() เมื่อเริ่มเกม มิฉะนั้น KeyListener จะไม่ได้รับเหตุการณ์
- **การปรับสมดุลความยาก:** ต้องปรับจำนวนปลา เป้าหมาย เวลา และความเร็วในแต่ละด่านให้ผ่านได้จริง (เช่น ด่าน 3 มีปลา 10 ตัวแต่ต้องจับ 9 ตัว)
- **การวาดภาพด้วยพิกัดเอง:** ตำแหน่งของภาพแต่ละส่วนต้องคำนวณเอง จึงต้องทดลองปรับหลายรอบ

3.2 จุดเด่นของโปรแกรม

- ออกแบบด้วย Abstract class (GameObject) ทำให้เพิ่มวัตถุชนิดใหม่ได้ง่าย เพียงสืบทอดและเขียน draw()
- ข้อมูลของแต่ละด่านรวมอยู่ในคลาส Level เดียว ปรับด่านหรือเพิ่มด่านใหม่ได้ง่าย
- ความยากเพิ่มขึ้นทีละขั้นผ่านความเร็วปลา เป้าหมาย และสิ่งกีดขวางที่อันตรายขึ้น (Rock ⟶ Trash ⟶ Mine)
- วาดกราฟิกทั้งหมดด้วยโค้ด ไม่พึ่งไฟล์ภาพ จึงพกพาง่าย รันได้จากไฟล์ .jar ไฟล์เดียว
- ใช้ enum (Obstacle.Type) กำหนดชนิดสิ่งกีดขวางอย่างปลอดภัยด้านชนิดข้อมูล
- มีข้อความแจ้งผลชั่วคราว (เช่น Caught Big Fish +20) ช่วยให้ผู้เล่นเข้าใจผลลัพธ์ทันที

3.3 คำแนะนำ / ข้อเสนอแนะ

อยากให้อาจารย์สอนช้าลงนิดหน่อยค่ะ และมีลงเฉลยแลปให้ค่ะ